渲染器在解析路径时对链接解引用，所以顺着类链接一路 cat 到设备属性完全透明。为把链接显式呈现，本工程补了一个最小 readlink(2)（SYS_readlink，仅 sysfs 实现了 readlink inode 钩子，其它文件系统如同 Linux 对非链接返回 EINVAL），ls 据此打印 name -> target。

笔记 与 Linux 的差异：本工程没有通用 VFS 符号链接与 lstat，链接只在 sysfs 渲染器内部解引用；readlink 返回绝对路径 /sys/devices/...，而 Linux 返回相对路径 ../../devices/...。此外 uevent 未走 netlink 用户态 udev，/sys/.../power 电源管理尚未实现。

7.2 上半部与下半部

中断处理程序运行时，本 CPU 的中断是关着的，而且它打断的可能是任何代码，所以不能睡眠、不能长时间运行。真正耗时的工作——解析 USB 报告、执行几十次 SDIO 命令、处理一个 TCP 段——要推迟到可以调度、可以睡眠的上下文中去做。项目实现了与 Linux 同名的 workqueue（kernel/workqueue.h）：

```
struct work_struct {
  void (*func)(struct work_struct *work);
  struct work_struct *next;
  int pending, running, rerun;
  struct workqueue *wq;
};
int  queue_work(struct workqueue *wq, struct work_struct *work);
int  queue_delayed_work(struct workqueue *wq, struct delayed_work *dw, uint64 delay);
void cancel_work_sync(struct work_struct *work);
```

每个 workqueue 有自己的内核线程（kworker），在 ps 中可以看到。几条原则贯穿所有驱动：

- 上半部只做三件事：确认中断来源、屏蔽或清除该中断、queue_work()；
- 每个设备使用私有的 work 和 workqueue，慢设备（如 SDIO 的 CMD53）不会阻塞别的下半部；
- 同一个 work 在执行期间再次被排队，只记一个 rerun 标志，不会无限堆积；
- 移除设备时先 cancel_work_sync()，等正在运行的 work 返回后才释放设备内存。

定时器中断每 10 ms 推进一次 delayed-work 时钟，TCP 重传、Wi-Fi 连接超时等都用 delayed work 实现，而不是在定时器中断里直接运行协议代码。网络接收还在 workqueue 之上实现了 NAPI 式的预算轮询：中断只调度一次，下半部每轮最多处理 16 个报文，没处理完就重新排队，处理完才重新打开中断。

7.3 控制台、UART 与 TTY

7.3.1 Mini UART

Mini UART 驱动（kernel/uart.c）在 probe() 中完成与早期汇编相同的初始化，再打开接收中断：

```
REGS_AUX->enables |= 1U;              // 使能 AUX 中的 Mini UART
REGS_AUX->mu_control = 0;             // 暂停收发
REGS_AUX->mu_ier = 0;
REGS_AUX->mu_lcr = 3;                 // 8 位数据
REGS_AUX->mu_iir = 0xc6;              // 清空 FIFO
REGS_AUX->mu_baud_rate = 270;         // 115200 @ core_freq 250 MHz
REGS_AUX->mu_ier = MU_IER_RX_ENABLE;  // 接收中断
REGS_AUX->mu_control = 3;             // 打开收发
```